

WIPRO PYTHON AUTOMATION

REPORT-4

Module 4: Robot Framework

Name: Debanjali De

Enrollment Number: 12023002001076

Year: 4th Year

Branch: Computer Science and Engineering

Wipro Training Programme, 2026

TABLE OF CONTENTS

- 1. Introduction
- 2. Module 4 – Robot Framework
 - 2.1 Introduction
 - 2.2 Script Running Options
 - 2.3 Make the Script More Readable
- 3. Module Summary
- 4. Learning Outcomes
- 5. Conclusion

1. Introduction

Robot Framework is an open-source automation framework designed around a readable, keyword-driven approach. It can be used to automate web applications and other systems by combining test cases, keywords, libraries, variables, data, and execution options.

This module introduces the Robot Framework environment, the basic structure of Robot Framework test cases, different ways to execute scripts, browser automation through Selenium Library, and techniques for making automation scripts more readable and maintainable.

The module also introduces data-driven testing, resource files, setup and teardown, page objects, and Gherkin-style readability so that automation scripts can be organized clearly for repeated execution.

2. Module 4 – Robot Framework

2.1 Introduction

Introduction to Robot Framework

Robot Framework is a keyword-driven automation framework in which test cases are written using readable keywords instead of embedding all automation logic directly into procedural code. A test suite can contain test cases, user-defined keywords, variables, settings, and imported libraries.

Structures to Set Up the Environment and Relevant Packages

A Robot Framework automation environment requires Python, Robot Framework, and the libraries needed by the automation project. For browser automation, Selenium Library is commonly used. A well-organized environment keeps test cases, resource files, test data, and supporting libraries separated so that the framework is easier to maintain.

Component	Purpose
Python	Provides the runtime environment for Robot Framework and supporting packages.

Robot Framework	Provides the test automation framework and execution engine.
Selenium Library	Provides keywords for browser-based automation.
Resource files	Store reusable keywords and variables.
Test data	Provides input values for data-driven test execution.

Robot Framework – RIDE

RIDE (Robot Integrated Development Environment) is an IDE designed to support Robot Framework development. It provides an interface for creating and editing Robot Framework files and can make test development easier for users who prefer an IDE-based workflow.

RIDE – Write Simple Test Case Using RIDE

A simple Robot Framework test case can be created in RIDE by defining a test case name and adding keywords under it. The keywords can open a browser, navigate to a URL, interact with page elements, and verify expected results. The readable tabular structure makes the sequence of actions easy to follow.

Keyword and Data Driven Test Cases, Writing and Executing Test Cases

Keyword-driven testing organizes reusable actions into keywords. Data-driven testing separates the test flow from input data so that the same test logic can be executed with different values.

Approach	Main idea
Keyword-driven	Reusable keywords represent actions or business operations.
Data-driven	The same test logic is executed with multiple sets of input data.
Combined approach	Reusable keywords perform actions while external or variable data controls the inputs.

Working with Command Line, Test Setup and Teardown

Robot Framework can be executed from the command line, allowing users to select test files, suites, tags, output locations, and other execution options. Setup and teardown keywords provide controlled actions before and after tests or suites.

Concept	Purpose
Test Setup	Runs before an individual test case.
Test Teardown	Runs after an individual test case.
Suite Setup	Runs before the test suite.
Suite Teardown	Runs after the test suite.

Working with Browsers Using Selenium Library

Selenium Library provides Robot Framework keywords for browser automation. It allows a test to open a browser, navigate to a URL, locate elements, enter text, click controls, and verify page content.

2.2 Script Running Options

Running Python, Command Window/Command Prompt, Batch File, Task Scheduler

Robot Framework tests can be started from different execution environments. Command-line execution is useful for development and CI workflows, batch files can group repeatable commands, and operating-system task schedulers can start automation at predefined times.

Execution option	Purpose
Python environment	Provides the environment in which Robot Framework and its libraries are installed.
Command Prompt / Terminal	Runs Robot Framework commands directly.

Batch file	Stores repeatable commands that can be executed together.
Task Scheduler	Allows scheduled execution of automation tasks.

Running on SauceLabs with Browsers

Sauce Labs is a cloud-based platform that can provide access to browsers and operating-system combinations for automated testing. A Robot Framework project can be configured to execute browser tests against a remote environment instead of only using a local browser.

Running & Scheduling on Jenkins

Jenkins is an automation server commonly used for continuous integration and continuous delivery workflows. Robot Framework tests can be triggered by Jenkins manually, after a build, or according to a configured schedule.

Running Single Test Case, Multiple Suites and Advanced Run Options

Robot Framework provides command-line options for controlling which tests are executed. Test selection can be based on files, suites, test names, or tags. This allows a complete suite, a selected group, or a single test case to be executed without running unrelated tests.

Execution scope	Use
Single test case	Useful for focused debugging or quick verification.
Multiple suites	Useful for executing related groups of tests.
Full suite	Useful for complete regression execution.
Tagged execution	Useful for selecting tests such as smoke or regression groups.

Procedural vs. Gherkin

Procedural test descriptions focus on the sequence of actions required to perform a test. Gherkin-style descriptions express behavior using a readable Given-When-Then structure. Robot Framework can be organized to improve behavioral readability even though its native syntax is keyword based.

Aspect	Procedural style	Gherkin-style
Focus	Sequence of actions	Expected behavior
Typical structure	Action-oriented steps	Given / When / Then
Readability	Technical execution flow	Business-readable behavior
Main benefit	Direct execution description	Clear communication of behavior

2.3 Make the Script More Readable

Introducing User-Defined Keywords

User-defined keywords are reusable groups of Robot Framework actions created by the test developer. They allow complex sequences to be represented by meaningful names and reduce repetition across test cases.

Break the Script into Keywords

Breaking a long test into smaller keywords makes each part easier to understand, debug, reuse, and maintain. For example, browser opening, login, navigation, and verification can be represented as separate logical keywords.

Move Keywords to Resource Files

Resource files store reusable keywords and variables separately from individual test suites. This promotes reuse and keeps test cases focused on the behavior being tested.

Adding Setup and Tear Down

Setup and teardown actions ensure that the test environment is prepared before execution and cleaned up afterward. This improves consistency between test runs and helps avoid dependency on the state left by a previous test.

Overview of Page Objects

The Page Object approach represents application pages or important page components as reusable objects or logical abstractions. The automation layer stores element-related information and common page actions in one place, reducing duplication when the same page is used by multiple tests.

Increase Readability Using Gherkin

Gherkin-style language can make test intent easier to understand by describing behavior through Given, When, and Then statements. This separates the business intent of a test from the technical implementation of the underlying keywords.

Variables and Data-Driven Automation

Variables allow values such as usernames, passwords, URLs, and test inputs to be reused without hard-coding them repeatedly. Data-driven automation extends this concept by running the same test logic with multiple sets of input data.

Technique	Benefit
Variables	Centralize reusable values and reduce hard-coded data.
Test data files	Keep input data separate from test logic.
Data-driven tests	Execute one test flow with multiple data sets.
Resource files	Reuse keywords and variables across suites.

Custom Keywords and Libraries

Custom keywords represent project-specific reusable actions, while libraries provide predefined functionality. Combining custom keywords with appropriate libraries allows a framework to remain readable while still providing the technical operations required for automation.

3. Module Summary

Module 4 introduces Robot Framework as a readable, keyword-driven automation framework. It covers environment setup, RIDE, keyword-driven and data-driven testing, command-line execution, setup and teardown, browser automation with Selenium Library, and multiple execution options. It then focuses on script readability through user-defined keywords, resource files, variables, page objects, Gherkin-style descriptions, and reusable libraries.

Section	Core focus
2.1 Introduction	Robot Framework basics, environment, RIDE, test cases, keywords, data-driven testing, setup/teardown, Selenium Library.
2.2 Script Running Options	Command line, batch files, Task Scheduler, Sauce Labs, Jenkins, test selection, procedural vs Gherkin.
2.3 Make the Script More Readable	User-defined keywords, reusable resources, setup/teardown, page objects, Gherkin, variables, data-driven automation, libraries.

4. Learning Outcomes

- Define Robot Framework and explain its keyword-driven automation approach.
- Understand the role of RIDE and the components required for a Robot Framework environment.
- Explain keyword-driven and data-driven test cases.
- Understand Robot Framework test setup, teardown, and browser automation using Selenium Library.
- Explain different methods of running and scheduling Robot Framework tests.
- Understand local and remote browser execution concepts, including Sauce Labs.
- Explain how Jenkins can be used to schedule or trigger automated tests.
- Differentiate procedural test descriptions from Gherkin-style descriptions.
- Create a conceptual structure using user-defined keywords and resource files.
- Understand variables, data-driven automation, page objects, custom keywords, and reusable libraries.

5. Conclusion

Robot Framework provides a readable structure for automation by combining test cases, keywords, variables, libraries, and execution options. Its support for Selenium Library makes it suitable for browser automation, while command-line execution, Jenkins, scheduling, and cloud browser platforms support different execution needs. User-defined keywords, resource files, page objects, variables, data-driven testing, and Gherkin-style readability help create automation scripts that are easier to understand, reuse, and maintain.